```
DELETE /oauth/clients/{client-id}
In order to delete clients.
axios.delete('/oauth/clients/' + clientId)
  .then(response => {
   // Code Logic
  });
```

Protecting Routes

The way to ensuring security in the routes is handled in a couple of ways. We will dive into each of them.

Middleware

An authentication guard included into Passport will verify access tokens for incoming requests. You simply need to include the **`auth:api`** middleware on all routes that need a valid access token after setting up the **`api`** guard to utilize the **`passport`** driver:

```
Route::get('/user', function () {
  // Code Logic
})->middleware('auth:api');
```

For each user provider type that your application authenticates—which may involve using completely distinct Eloquent models—you will probably need to specify a guard configuration. By doing this, you may safeguard requests meant for particular user providers. For instance, the **`config/auth.php`** configuration file has the following guard settings:

```
'api' => [
  'driver' => 'passport',
  'provider' => 'users',
],
'api-customers' => [
  'driver' => 'passport',
  'provider' => 'customers',
],
```

The **`api-customers`** guard, which verifies incoming requests using the **`customers`** user provider, will be used in the subsequent route:

```
Route::get('/customer', function () {
```